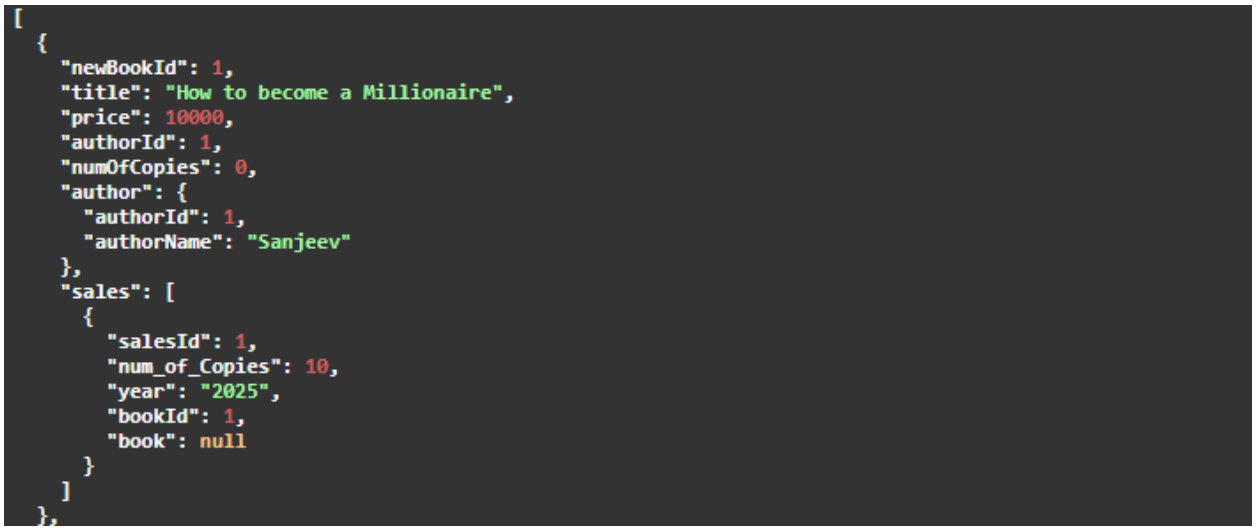
**DAILY TASK - 14**

**NAME : SANJEEV N**
**DATE : 25-08-2025**





**BOOKCONTROLLER.CS**

```
using day10.Models;
using day10.Services.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace day10.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class BookController : Controller
    {
        private readonly IBookService _bookService;

        public BookController(IBookService bookService)
        {
            _bookService = bookService;
        }

        [HttpGet("GetallBooks")]
        public IActionResult GetAllBooks() => Ok(_bookService.GetAllBooks());

        [HttpGet("getById")]
```

```
public IActionResult GetById(int id)
{
    var book = _bookService.GetBookById(id);
    return book == null ? NotFound() : Ok(book);
}

[Authorize]
[HttpPost("AddBooks")]
public IActionResult AddBooks(Book book)
{
    _bookService.AddBook(book);
    return Ok("Books Added");
}

[Authorize(Roles = "Admin")]
[HttpDelete("DeleteBook")]
public IActionResult DeleteBook(int id)
{
    _bookService.DeleteBook(id);
    return Ok("Deleted");
}

[HttpPost("AddNewAuthor")]
public IActionResult AddNewAuthor(Author author)
{
    _bookService.AddNewAuthor(author);
    return Ok("Author Added");
}

[HttpPost("AddNewBooks")]
public IActionResult AddNewBooks(bnaEntry bnaentry) =>
    Ok(_bookService.AddNewBook(bnaentry));

[HttpPost("InsertSales")]
public IActionResult AddSalesInfo(sentry entry) =>
    Ok(_bookService.InsertSales(entry));

[HttpGet("FetchAllNewBooks")]
public IActionResult FetchAllBooks() =>
    Ok(_bookService.FetchAllNewBooks());

[HttpGet("FetchAllNewBooksauthor")]
public IActionResult FetchAllNewBooksByAuthor(string sName) =>
    Ok(_bookService.FetchAllNewBooksByAuthor(sName));
```

```
        [HttpGet("getbyjoin")]
        public IActionResult GetAllBooksJoin(string author_name) =>
            Ok(_bookService.GetAllBooksJoin(author_name));

        [HttpGet("getbyauthorjoin")]
        public IActionResult GetAllBooksAuthorJoin() =>
            Ok(_bookService.GetAllBooksAuthorJoin());

        [HttpGet("GetSalesByAuthor")]
        public IActionResult GetSalesByAuthor(string author_name) =>
            Ok(_bookService.GetSalesByAuthor(author_name));

        [HttpPost("GetBooksOrderedByPrice")]
        public IActionResult GetBooksOrderedByPrice([FromQuery] bool ascending) =>
            Ok(_bookService.GetBooksOrderedByPrice(ascending));

        [HttpGet("GetTop5Books")]
        public IActionResult GetTop5Books() =>
            Ok(_bookService.GetTop5Books());

        [HttpGet("GetCostDetails")]
        public IActionResult GetCostDetails() =>
            Ok(_bookService.GetCostDetails());
    }
}
```

**BOOKSERVICE.CS**

```
using day10.Context;
using day10.Models;
using day10.Services.Interfaces;
using Microsoft.EntityFrameworkCore;

namespace day10.Services
{
    public class BookService : IBookService
    {
        private readonly AppDbContext _context;
        public BookService(AppDbContext context)
        {
            _context = context;
        }
```

```
    public IEnumerable<Book> GetAllBooks() => _context.Books.ToList();

    public Book? GetBookById(int id) =>
        _context.Books.FirstOrDefault(b => b.Id == id);

    public void AddBook(Book book)
    {
        _context.Books.Add(book);
        _context.SaveChanges();
    }

    public void DeleteBook(int id)
    {
        var book = _context.Books.FirstOrDefault(b => b.Id == id);
        if (book != null)
        {
            _context.Books.Remove(book);
            _context.SaveChanges();
        }
    }

    public void AddNewAuthor(Author author)
    {
        _context.authors.Add(author);
        _context.SaveChanges();
    }

    public string AddNewBook(bnaEntry bnaentry)
    {
        var author = _context.authors.FirstOrDefault(x => x.AuthorName ==
bnaentry.AuthorName);
        if (author == null) return "Give valid author name";

        NewBook newBook = new()
        {
            Title = bnaentry.BookTitle,
            Price = bnaentry.Price,
            AuthorId = author.AuthorId
        };
        _context.NewBooks.Add(newBook);
        _context.SaveChanges();
        return "New Book Added";
    }
```

```
        public string InsertSales(sentry entry)
        {
            var book = _context.NewBooks.FirstOrDefault(x => x.Title == entry.Book_Name);
            if (book == null) return "No book";

            Sales sales = new()
            {
                BookId = book.NewBookId,
                Num_of_Copies = entry.Num_of_Copies,
                Year = entry.Year
            };
            _context.booksales.Add(sales);
            _context.SaveChanges();
            return "Sales Inserted";
        }

        public IEnumerable<object> FetchAllNewBooks() =>
            _context.NewBooks.Include(x => x.author).Include(x => x.Sales).ToList();

        public IEnumerable<object> FetchAllNewBooksByAuthor(string authorName) =>
            _context.NewBooks
                .Where(y => y.author.AuthorName == authorName)
                .Select(x => new { x.Title, Rate = x.Price, Author = x.author.AuthorName })
                .ToList();

        public IEnumerable<object> GetAllBooksJoin(string authorName) =>
            _context.booksales.Join(_context.NewBooks,
                sales => sales.BookId,
                books => books.NewBookId,
                (sale, book) => new { Name = book.Title, copies = sale.Num_of_Copies, sale.Year,
author = book.author.AuthorName })
            .Where(y => y.author == authorName)
            .OrderBy(x => x.Year)
            .ToList();

        public IEnumerable<object> GetAllBooksAuthorJoin()
        {
            var joined = _context.booksales.Join(_context.NewBooks,
                sales => sales.BookId,
                books => books.NewBookId,
                (sale, book) => new { Name = book.Title, copies = sale.Num_of_Copies, sale.Year,
author = book.author.AuthorName, bookId = book.NewBookId });
```

```
            var result = _context.NewBooks.GroupJoin(joined,
                book => book.NewBookId,
                prev => prev.bookId,
                (book, grp) => new { book.Title, book.Price, grp = grp.ToList() });

            return result;
        }

        public IEnumerable<object> GetSalesByAuthor(string authorName) =>
            _context.booksales.Where(x => x.Book.author.AuthorName == authorName)
            .Select(y => new { y.Book.Title, y.Year, y.Num_of_Copies, y.Book.Price,
y.Book.author.AuthorName })
            .ToList();

        public IEnumerable<Book> GetBooksOrderedByPrice(bool ascending) =>
            ascending
                ? _context.Books.OrderBy(b => b.Price).ToList()
                : _context.Books.OrderByDescending(b => b.Price).ToList();

        public IEnumerable<Book> GetTop5Books() =>
            _context.Books.OrderByDescending(b => b.Price).Take(5).ToList();

        public object GetCostDetails() => new
        {
            TotalPrice = _context.Books.Sum(a => a.Price),
            AvgPrice = _context.Books.Average(a => a.Price)
        };
    }
}
```

**IBOOKSERVICE.CS**

```
using day10.Context;
using day10.Models;
using day10.Services.Interfaces;
using Microsoft.EntityFrameworkCore;

namespace day10.Services
{
    public class BookService : IBookService
    {
        private readonly AppDbContext _context;
        public BookService(AppDbContext context)
```

```
    {
        _context = context;
    }

    public IEnumerable<Book> GetAllBooks() => _context.Books.ToList();

    public Book? GetBookById(int id) =>
        _context.Books.FirstOrDefault(b => b.Id == id);

    public void AddBook(Book book)
    {
        _context.Books.Add(book);
        _context.SaveChanges();
    }

    public void DeleteBook(int id)
    {
        var book = _context.Books.FirstOrDefault(b => b.Id == id);
        if (book != null)
        {
            _context.Books.Remove(book);
            _context.SaveChanges();
        }
    }

    public void AddNewAuthor(Author author)
    {
        _context.authors.Add(author);
        _context.SaveChanges();
    }

    public string AddNewBook(bnaEntry bnaentry)
    {
        var author = _context.authors.FirstOrDefault(x => x.AuthorName ==
bnaentry.AuthorName);
        if (author == null) return "Give valid author name";

        NewBook newBook = new()
        {
            Title = bnaentry.BookTitle,
            Price = bnaentry.Price,
            AuthorId = author.AuthorId
        };
        _context.NewBooks.Add(newBook);
```

```
        _context.SaveChanges();
        return "New Book Added";
    }

    public string InsertSales(sentry entry)
    {
        var book = _context.NewBooks.FirstOrDefault(x => x.Title == entry.Book_Name);
        if (book == null) return "No book";

        Sales sales = new()
        {
            BookId = book.NewBookId,
            Num_of_Copies = entry.Num_of_Copies,
            Year = entry.Year
        };
        _context.booksales.Add(sales);
        _context.SaveChanges();
        return "Sales Inserted";
    }

    public IEnumerable<object> FetchAllNewBooks() =>
        _context.NewBooks.Include(x => x.author).Include(x => x.Sales).ToList();

    public IEnumerable<object> FetchAllNewBooksByAuthor(string authorName) =>
        _context.NewBooks
            .Where(y => y.author.AuthorName == authorName)
            .Select(x => new { x.Title, Rate = x.Price, Author = x.author.AuthorName })
            .ToList();

    public IEnumerable<object> GetAllBooksJoin(string authorName) =>
        _context.booksales.Join(_context.NewBooks,
            sales => sales.BookId,
            books => books.NewBookId,
            (sale, book) => new { Name = book.Title, copies = sale.Num_of_Copies, sale.Year,
author = book.author.AuthorName })
        .Where(y => y.author == authorName)
        .OrderBy(x => x.Year)
        .ToList();

    public IEnumerable<object> GetAllBooksAuthorJoin()
    {
        var joined = _context.booksales.Join(_context.NewBooks,
            sales => sales.BookId,
            books => books.NewBookId,
```

```
            (sale, book) => new { Name = book.Title, copies = sale.Num_of_Copies, sale.Year,
author = book.author.AuthorName, bookId = book.NewBookId });

        var result = _context.NewBooks.GroupJoin(joined,
            book => book.NewBookId,
            prev => prev.bookId,
            (book, grp) => new { book.Title, book.Price, grp = grp.ToList() });

        return result;
    }

    public IEnumerable<object> GetSalesByAuthor(string authorName) =>
        _context.booksales.Where(x => x.Book.author.AuthorName == authorName)
        .Select(y => new { y.Book.Title, y.Year, y.Num_of_Copies, y.Book.Price,
y.Book.author.AuthorName })
        .ToList();

    public IEnumerable<Book> GetBooksOrderedByPrice(bool ascending) =>
        ascending
            ? _context.Books.OrderBy(b => b.Price).ToList()
            : _context.Books.OrderByDescending(b => b.Price).ToList();

    public IEnumerable<Book> GetTop5Books() =>
        _context.Books.OrderByDescending(b => b.Price).Take(5).ToList();

    public object GetCostDetails() => new
    {
        TotalPrice = _context.Books.Sum(a => a.Price),
        AvgPrice = _context.Books.Average(a => a.Price)
    };
  }
}
```

**PROGRAM.CS**

```
using day10.Context;
using day10.Security;
using day10.Services;
using day10.Services.Interfaces;

using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
```

```
using System.Text;
using System.Security.Claims;

var builder = WebApplication.CreateBuilder(args);

var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString));

builder.Services.Configure<JWTOptions>(builder.Configuration.GetSection("JwtSettings"));

var jwtSettings = builder.Configuration.GetSection("JwtSettings");
var key = Encoding.UTF8.GetBytes(jwtSettings["Key"]);

builder.Services.AddScoped<IBookService, BookService>();

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = jwtSettings["Issuer"],
        ValidAudience = jwtSettings["Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(key),
        ClockSkew = TimeSpan.Zero,
        RoleClaimType = ClaimTypes.Role,
        NameClaimType = ClaimTypes.NameIdentifier
    };
});

builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.ReferenceHandler =
            System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;
    });
```

```
builder.Services.AddHttpClient();
builder.Services.AddEndpointsApiExplorer();


builder.Services.AddSwaggerGen(options =>
{
   options.AddSecurityDefinition("Bearer", new
Microsoft.OpenApi.Models.OpenApiSecurityScheme
   {
      Name = "Authorization",
      Type = Microsoft.OpenApi.Models.SecuritySchemeType.ApiKey,
      Scheme = "Bearer",
      BearerFormat = "JWT",
      In = Microsoft.OpenApi.Models.ParameterLocation.Header,
      Description = "Enter 'Bearer' [space] and then your valid token.\r\n\r\nExample: \"Bearer
eyJhbGciOiJIUzI1NiIs...\""
   });

   options.AddSecurityRequirement(new
Microsoft.OpenApi.Models.OpenApiSecurityRequirement
   {
      {
         new Microsoft.OpenApi.Models.OpenApiSecurityScheme
         {
            Reference = new Microsoft.OpenApi.Models.OpenApiReference
            {
               Type = Microsoft.OpenApi.Models.ReferenceType.SecurityScheme,
               Id = "Bearer"
            }
         },
         new string[] {}
      }
   });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
   app.UseSwagger();
   app.UseSwaggerUI();
}

app.UseHttpsRedirection();
```

```
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();
```